



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 2 – Overloading and Inheritance
Topic – 2.7 Ambiguity in Multiple Inheritance and use of Virtual Base Class

Prof. V.N.Nirgude
Assistant Professor

E-mail :

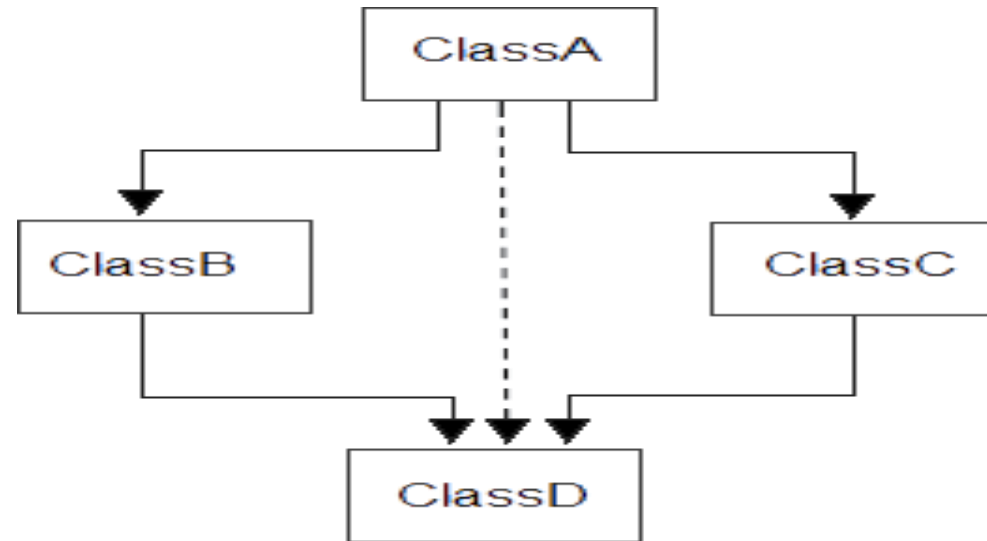
nirgudevikascomp@sanjivani.org.in

Contact No: 9975240215

A derived class with two base classes and these two base classes have one common base class is called **multipath inheritance**.

C++ Ambiguity:

Ambiguity in C++ occurs when a derived class has two base classes and these two base classes have one common base class. Consider the following figure:





Example of occurrence of C++ ambiguity

```
class A
{
    public:
    int a;
};
class B : public A
{
    public:
    int b;
};
class C : public A
{
    public:
    int c;
};
```

```
class D : public B, public C
{
    public:
    int d;
};
int main()
{
    D obj;
    //obj.a = 10;    //Statement 1, Error
    obj.B::a = 10;   //Statement 2
    obj.C::a = 100;  //Statement 3
    obj.b = 20;
    obj.c = 30;
    obj.d = 40;
    cout<< "a from Class B : "<<obj.B::a;
    cout<< "a from Class C : "<<obj.C::a;
```

```
    cout<< "B : "<< obj.b;
    cout<< "C : "<< obj.c;
    cout<< "D : "<< obj.d;
}
Output :
    a from Class B : 10
    a from Class C : 100
    B : 20
    C : 30
    D : 40
```



- In the above example, both Class B & Class C inherit Class A, they both have single copy of Class A. However Class D inherit both Class B & Class C, therefore Class D have two copies of Class A, one from Class B and another from Class C.
- If we need to access the data member a of Class A through the object of Class D, we must specify the path from which a will be accessed, whether it is from Class B or Class C, because compiler can't differentiate between two copies of Class A in Class D.

Two ways to avoid this ambiguity:

- Using scope resolution operator
- Using Virtual Base Class



Using scope resolution operator

Using scope resolution operator we can manually specify the path from which data member **a** will be accessed, as shown in statement 2 and 3, in the below example:

```
obj.B::a = 10;    //Statement 2
```

```
obj.C::a = 100;   //Statement 3
```

Note : Still, there are two copies of Class A in Class D.

To remove multiple copies of **Class A** in **Class D**, we must inherit **Class A** in **Class B** and **Class C** as **Virtual Base Class**.



Using Virtual Base Class

```
class A
{
    public:
    int a;
};
class B : virtual public A
{
    public:
    int b;
};
class C : virtual public A
{
    public:
    int c;
};
```

```
class D : public B, public C
{
    public:
    int d;
};
int main()
{
    D obj;
    obj.a = 10;    //Statement 2
    obj.a = 100;   //Statement 3
    obj.b = 20;
    obj.c = 30;
    obj.d = 40;
    cout<< "A : "<< obj.a;
    cout<< "B : "<< obj.b;
```

```
    cout<< "C : "<< obj.c;
    cout<< "D : "<< obj.d;
}
Output :
```

A : 100
B : 20
C : 30
D : 40

According to the above example, Class D have only one copy of Class A therefore statement 3 will overwrite the value of a, given at statement 2.



Thank You..